

Lab_Day02-Securing Accounts

Lab Objectives

By the end of this lab session, students will be able to:

- Analyse Cryptographic Avalanche Effects: Demonstrate how a single-bit change in plaintext structurally alters hash outputs across different hashing algorithms (MD5, SHA-1, SHA-256), and identify system hashing frameworks via /etc/shadow and standard prefixes.
- Execute High-Performance Password Cracking: Audit and break insecure passwords by leveraging automated dictionary tools (Hashcat) and online pre-computed lookup tables (CrackStation).
- Evaluate Threat Horizons & Data Exposure: Quantify personal attack surfaces using public breach repositories (HaveIBeenPwned) and ecosystem diagnostics (Google Password Checkup) to identify risks from credential stuffing and password reuse.
- Industry grade password managers: Critically compare enterprise password managers based on source licensing, underlying cryptography (AES-256, Argon2id), and legal jurisdictions to defend a chosen security posture.
- Operationalize Personal Identity Hardening: Deploy a secure, production-grade password vault protected using MFA.

Lab Tasks

TASK 1: PASSWORD CRACKING	2
TASK 1A: BASICS OF HASH FUNCTIONS.....	2
TASK 1B: HASHCAT (PASSWORD CRACKING):	2
TASK 2: PASSWORD AUDIT	3
TASK 2A: CHECK IF YOU HAVE BEEN PWNED:	3
TASK 2B: GOOGLE PASSWORD AUDIT	4
TASK 3: PASSWORD MANAGER	4



Task 1: Password Cracking

One of the most effective ways an attacker can compromise a system or elevate their privileges is by acquiring valid user credentials. However, modern operating systems and web applications do not store passwords in plain text; they store them as cryptographic hashes. To an ethical hacker, a database dump of hashes is useless unless those hashes can be reversed back into plaintext passwords.

Task 1a: Basics of Hash functions

1. Create two files file1.txt and file2.txt with contents T and U respectively.
2. Check their contents using hexdump and xxd to note only one bit difference.
3. Check md5sum, sha1sum and sha256sum of both files; and note their difference. Also note same output sizes.
4. Generate a salted password hash using SHA-512 (`openssl passwd --help`)
5. [CrackStation](#) and [Hashes.com](#) provide online rainbow tables. Crack the following hash:
5b31f93c09ad1d065c0491b764d04933 -> [REDACTED]
6. Check hashed passwords on your Linux machine (in shadow).

Commands for above tasks:

```
$ echo 'T' > file1.txt
$ echo 'U' > file2.txt

$ hexdump -C file1.txt
$ hexdump -C file2.txt

$ xxd -b file1.txt
$ xxd -b file2.txt

$ md5sum *.txt
$ sha1sum *.txt
$ sha256sum *.txt

$ openssl passwd -6

$ sudo cat /etc/shadow
```

7. Check various hash formats and prefixes at [refixes](#) is the [Hashcat Example Hashes](#) page.
 - a. What hashing algorithm is used in Cisco-IOS if it starts with \$9\$? [REDACTED]
 - b. What's the Hash-Mode listed for Cisco-ASA MD5? [REDACTED]

Task 1b: Hashcat (password cracking):

Hashcat uses the following basic syntax: `hashcat -m <hash_type> -a <attack_mode> hashfile wordlist`, where:



- **-m <hash_type>** specifies the hash-type in numeric format. For example, -m 1000 is for NTLM. Check the official documentation (man hashcat) and example page to find the hash type code to use.
- **-a <attack_mode>** specifies the attack-mode. For example, -a 0 is for straight, i.e., trying one password from the wordlist after the other.
- **hashfile** is the file containing the hash you want to crack.
- **wordlist** is the security word list you want to use in your attack.

Crack the hash: \$2a\$06\$7yoU3Ng8dHTXphAg913cyO6Bjs3K5lBnwq5FJyA6d01pMSrddr1ZG

1. Identify the hash type and corresponding Hashcat mode:

```
hashid -m '$2a$06$7yoU3Ng8dHTXphAg913cyO6Bjs3K5lBnwq5FJyA6d01pMSrddr1ZG'
```

2. Create the hash file:

```
echo '$2a$06$7yoU3Ng8dHTXphAg913cyO6Bjs3K5lBnwq5FJyA6d01pMSrddr1ZG' > hash1.txt
```

3. Run the attack:

```
hashcat -m 3200 hash1.txt /usr/share/wordlists/rockyou.txt
```

(note: if you have zipped file, unzip it first: `sudo gunzip /usr/share/wordlists/rockyou.txt.gz`)

Password: XXXXXXXXXX

Crack the hash: 9eb7ee7f551d2f0ac684981bd1f1e2fa4a37590199Ta636753efe614d4db30e8e1

Password: XXXXXXXXXX

Task 2: Password Audit

Security starts with your own digital footprint. A single compromised credential can lead to a supply-chain breach, data exfiltration, or cloud infrastructure takeovers.

This task is a hands-on operational audit of your own passwords. You will analyse your credential hygiene, evaluate cryptographic vulnerability risks (like password reuse), and take your remediation steps.

Task 2a: Check if you have been pwned:

1. Open your web browser (on your host machine or Kali VM) and navigate to <https://haveibeenpwned.com/>
2. Enter an email address you have owned for several years (or use a known public test email).
3. **Analyze the Results:** Review the list of breaches associated with the account.
4. **Identify the Data Vectors:** Note the specific types of data exposed in each breach (e.g., Passwords, IP addresses, Physical addresses, Phone numbers).

Observe how a breach on a low-security forum can expose the password you might have reused on a high-security banking site.

Task 2b: Google Password Audit

1. Open your web browser (on your host machine or Kali VM) and navigate to <https://passwords.google.com/checkup/start?ep=1>
2. Check your Google Password vault backup report:
 - **Compromised Passwords:** How many of your saved credentials have appeared in known data breaches? These are vulnerable to **Credential Stuffing** attacks, where malicious actors use automated scripts to test leaked username/password pairs across thousands of sites.
 - **Reused Passwords:** How many accounts share the same or similar root password? This exponentially increases your attack surface. If your low-security forum account is breached, your high-security GitHub or banking account is now at risk.
 - **Weak Passwords:** How many passwords rely on predictable character sets, short lengths, or dictionary words?

Task 3: Password Manager

Relying on a browser's built-in credential storage or memory for passwords is a significant single point of failure. Industry-grade protection requires a dedicated, **Zero-Knowledge Architecture** password manager.

As a computer science student, you should not blindly trust marketing claims. In this lab, you will audit the licenses, underlying cryptography, and financial models of the industry's top 3 tools, defend your architectural choice, and operationalize your defense by deploying your chosen vault.

Review the foundational specifications of the top three platforms leading the industry: **Bitwarden**, **1Password**, and **Proton Pass**.

Vector	Bitwarden	1Password	Proton Pass
License Model	Open Source (GPLv3 / Commercial)	Proprietary (Closed Source)	Open Source (GPLv3)
Cost (Free Tier)	Generous: Unlimited passwords & devices.	None: Paid only (30-day trial).	Good: Unlimited passwords/devices; limits advanced aliases.
Core Crypto Engine	AES-256 bit encryption via PBKDF2 SHA-256 or Argon2id.	AES-GCM-256 authenticated encryption.	AES-GCM-256 bit encryption.
Key Architectural Feature	Self-hosting capable via Docker (Vaultwarden).	Secret Key: An additional 128-bit local high-entropy string required for auth.	Integrated SimpleLogin email aliasing to mask identities.

Jurisdiction	United States (Subject to US warrants/gag orders).	Canada (Five Eyes intelligence sharing network).	Switzerland (Strict Federal Act on Data Protection - FADP).
---------------------	--	--	---

Analyze the data above and justify which password manager is optimal for your specific threat model. Consider trade-offs like:

- *Do you prioritize a Swiss jurisdiction and identity masking, or the ability to review and self-host the open-source code?*
- *Does 1Password's proprietary dual-key architecture (Master Password + Secret Key) offset its lack of a free tier?*

Create an account with your chosen platform and implement structural security controls:

1. **Master Password Generation:** Construct a master passphrase using a minimum 4-word Diceware method (e.g., combining four random words to maximize entropy). **Do not use a password generator or dictionary phrases.**
2. **Zero-Knowledge Check:** Confirm that your provider has no technical means of resetting your master password from their server-side database.
3. **Deploy Two-Factor Authentication (2FA):** Immediately protect your new vault using a TOTP authenticator app or hardware security key (e.g., YubiKey). *Never allow the password manager to store its own 2FA key.*